

Bug Report Ticket

GCD Module (UPC MIRI – PV)

Dan Hagen

June 20, 2026

Ticket	GCD-001
Component	rtl/gcd.sv — FSM convergence (RUN state)
Severity	Major (functional timing)
Status	Open
Found by	UVM regression (smoke, directed, random)

Summary

Every general-case GCD computation produces the *correct result* but asserts `out_valid one clock cycle too late`, violating the latency specification. Early-exit operands ($a = 0$, $b = 0$, $a = b$) are unaffected.

Steps to Reproduce

`make regress` (seeds 1 / 7 / 42). Failures appear in all three tests.

Observed vs. Expected

848 of 1025 scoreboard checks fail. The result value always matches the reference; the measured latency is always `expected + 1` for any operand pair that enters the RUN state. Representative cases:

Operands	gcd.out (obs/exp)	Latency obs	Latency exp	Result
gcd(6, 9)	3 / 3	4	3	value OK, +1 cycle
gcd(3, 5)	1 / 1	5	4	value OK, +1 cycle
gcd(65535, 1)	1 / 1	65536	65535	value OK, +1 cycle
gcd(0, 5)	5 / 5	1	1	pass (early-exit)

Detection

- **Scoreboard latency check (RQ-L02)** — reports `delay = N+1`, Expected delay = N on every general-case transaction.
- **Assertion A_RUN_CONVERGE** (`tb_gcd.sv:1324`, `(a_next==b_next && state==RUN) | => state==DONE`) — fired 848 times.

Value-correctness, handshake, and reset checks all pass; the defect is purely temporal, so a value-only testbench would not have caught it.

Root Cause

The RUN→DONE convergence test compares the *registered* operands instead of the *combinational next-cycle* operands:

```
Buggy    if (a_reg == b_reg)
Correct  if (a_next == b_next)
```

Spec §2.4 requires the termination check to be evaluated on the next-cycle values; comparing the registered values detects convergence one cycle late, so the FSM stays in RUN for one extra cycle before entering DONE.

Proposed Fix

rtl/gcd.sv, RUN-state convergence test: change `if (a_reg == b_reg)` back to `if (a_next == b_next)`. One-token change; restores §2.5 latency ($1+N$ cycles) with no datapath impact.